

Exercise 2: Test Table Practice

Boolean Algebra Expression

Create a truth table and test table for this expression: $A \wedge \neg(B \wedge C)$.

Truth Table/Test Table (column R makes this a Test Table also)

A	B	C	Q (expected)	R (actual)	Subexpression (optional)
0	0	0	0	0	-
0	0	1	0	0	-
0	1	0	0	0	-
0	1	1	0	0	-
1	0	0	1	1	$A \wedge \sim B \wedge \sim C$
1	0	1	1	1	$A \wedge \sim B \wedge C$
1	1	0	1	1	$A \wedge B \wedge \sim C$
1	1	1	0	0	-

[Optional for extra credit] Truth Table Boolean Algebra Expression

$$(A \wedge \sim B \wedge \sim C) \vee (A \wedge \sim B \wedge C) \vee (A \wedge B \wedge \sim C)$$

[Optional for extra credit] Simplification

Original expression: $(A \wedge \sim B \wedge \sim C) \vee (A \wedge \sim B \wedge C) \vee (A \wedge B \wedge \sim C)$

Step 1: Group the two terms using the Associative law

Group 1 $(A \wedge \sim B \wedge \sim C) \vee (A \wedge \sim B \wedge C)$

Group 2 $(A \wedge B \wedge \sim C)$

Step 2: Factor out the common terms using the distributive law. Remaining terms go in parentheses

$$A \wedge \sim B \wedge (\sim C \vee C)$$

Step 3: work inside the parenthesis using the complement law

$$(\sim C \vee C) = 1$$

Step 4: Result of complement law

Tyler Velez

$A \wedge \sim B \wedge 1$

Step 5: use the identity law

$A \wedge \sim B \wedge 1$ becomes $(A \wedge \sim B)$

Step 6: Rewrite

$(A \wedge \sim B) \vee (A \wedge B \wedge \sim C)$

Step 7 factor out the common terms with the distributive property

$A \wedge (\sim B \vee (B \wedge \sim C))$

Step 8: Result of distributive law

$A \wedge ((\sim B \vee B) \wedge (\sim B \vee \sim C))$

Step 9: Work inside the parentheses, using the complement law

$\sim B \vee B = 1$

Step 10: Result of using the complement law

$A \wedge (1 \wedge (\sim B \vee \sim C))$

Step 11: Use the identity law again

$(1 \wedge (\sim B \vee \sim C))$ becomes $(\sim B \vee \sim C)$

Step 12 rewrite everything fully simplified

$A \wedge (\sim B \vee \sim C)$

Python Function

```
def circuit(a, b, c):
```

```
    """emulates  $A \wedge \neg(B \wedge C)$ """
```

```
    gate1 = gate_and(b, c) #  $B \wedge C$ 
```

```
    gate2 = gate_not(gate1) #  $\neg(B \wedge C)$ 
```

```
    gate3 = gate_and(a, gate2) #  $A \wedge \neg(B \wedge C)$ 
```

```
    return gate3
```

```
print(circuit(0, 1, 1))
```